SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)**
Weightage:	**30%**	**Total Marks:**	**25**
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	S Susmitha	**Reg.No.:**	192571068
Department:	BTech CSE BioSciences	**Date:**	29.08.2026

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I S Susmitha (192571068) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 2

Name	Register Number	Course Code	Course
S Susmitha	192571068	CSA1202	Computer Architecture

Question 1 : DMA-Based Data Transfer Algorithm

Problem Understanding

Input: source address (I/O device buffer), destination address (memory block), block count (number of words to transfer), transfer mode. Output: data block moved from I/O device to memory with a completion interrupt raised. Constraint: CPU involvement must be minimized during the actual data movement, and interrupts arriving mid-transfer must not corrupt the transfer.

Pseudocode

```
ALGORITHM DMA_Transfer(src_addr, dest_addr, block_count, mode)
BEGIN
    // Step 1: CPU initializes DMA controller registers
    SET DMA.source_register = src_addr
    SET DMA.dest_register   = dest_addr
    SET DMA.count_register  = block_count
    SET DMA.mode_register   = mode        // read / write / block / burst

    // Step 2: CPU hands off control and continues other work
    ISSUE DMA_REQUEST TO DMA_Controller
    CPU CONTINUES executing next instruction   // CPU not blocked

    // Step 3: DMA controller performs transfer autonomously
    WHILE DMA.count_register > 0 DO
        REQUEST bus ownership (HOLD signal to CPU)
        WAIT UNTIL HOLD_ACK received FROM CPU

        TRANSFER one word: I/O_device <-> Memory[DMA.dest_register]

        DMA.source_register = DMA.source_register + 1
        DMA.dest_register   = DMA.dest_register + 1
        DMA.count_register  = DMA.count_register - 1

        IF external_interrupt_pending() AND mode == CYCLE_STEALING THEN
            RELEASE bus temporarily TO CPU   // minimize CPU stall
        END IF
    END WHILE

    // Step 4: Signal completion
    RAISE INTERRUPT(DMA_COMPLETE)
    RETURN control OF bus TO CPU
END
```

```
ISR DMA_Complete_Handler()
BEGIN
    VERIFY DMA.status_register == SUCCESS
    IF error_flag SET THEN
        LOG error AND RETRY transfer (bounded retries)
    ELSE
        UPDATE memory-mapped buffer descriptors
        CLEAR interrupt flag
    END IF
END
```

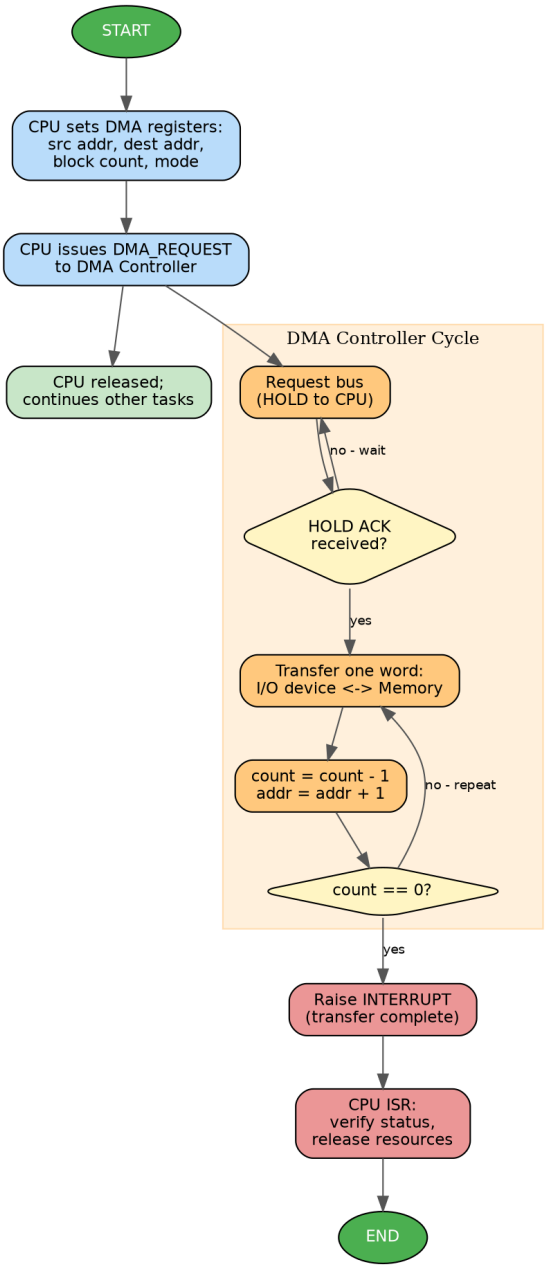

Fig 1.1 — DMA controller data transfer with interrupt-based completion signalling

Design Justification

The WHILE loop drives the block transfer while an IF checks for error/interrupt conditions, and a separate ISR (function) handles completion — satisfying the control-structure requirement (loop, condition, function). Cycle-stealing is included to show how CPU involvement is minimized even for long transfers, and the algorithm terminates correctly for block_count = 0 and reports failure through the status register, covering edge cases.

Question 2 : Vectored Interrupt Handling System

Problem Understanding

Input: interrupt requests (IRQ) from multiple devices, each carrying a priority level and a vector address. Output: correct ISR executed for the highest-priority pending device, with CPU context preserved. Constraint: lower-priority interrupts must not block or corrupt the servicing of a higher-priority one.

Pseudocode

```
ALGORITHM Vectored_Interrupt_Handler(device_list)
BEGIN
    pending_queue = EMPTY_PRIORITY_QUEUE

    WHILE system_running DO
        // Step 1: collect all active interrupt lines
        FOR EACH device IN device_list DO
            IF device.IRQ_line == ACTIVE THEN
                ENQUEUE(pending_queue, device, device.priority)
            END IF
        END FOR

        IF pending_queue IS NOT EMPTY THEN
            WHILE pending_queue IS NOT EMPTY DO
                // Step 2: pick highest-priority pending interrupt
                current_device = DEQUEUE_MAX_PRIORITY(pending_queue)

                // Step 3: save CPU state
                SAVE_CONTEXT(PC, registers, flags)

                // Step 4: fetch vector and dispatch
                vector_addr = VECTOR_TABLE[current_device.id]
                CALL ISR AT vector_addr WITH current_device

                CLEAR current_device.IRQ_line

                // Step 5: nested higher-priority check
                IF new_interrupt_arrived() AND
                   new_interrupt.priority > current_device.priority THEN
                    ENQUEUE(pending_queue, new_interrupt, new_interrupt.priority)
                END IF
            END WHILE
            RESTORE_CONTEXT(PC, registers, flags)
        END IF
    END WHILE
END

FUNCTION ISR(device)
BEGIN
    READ device.status_register
    SERVICE device request (data transfer / error handling)
    WRITE acknowledgement TO device.control_register
    RETURN
END
```

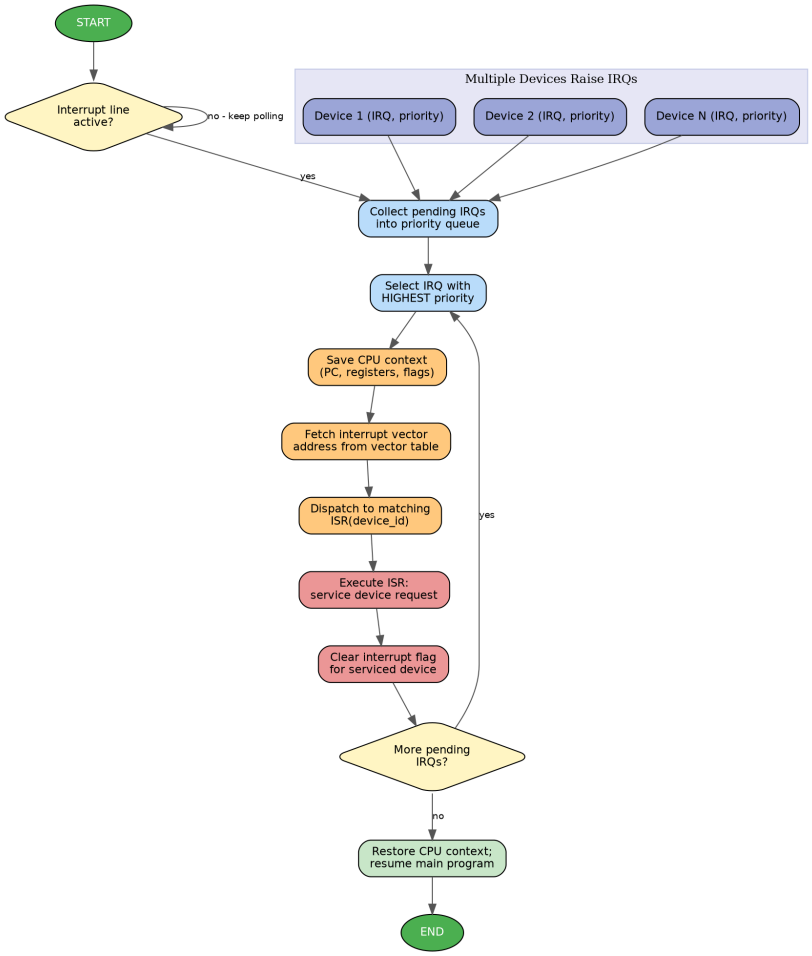

Fig 2.1 — Priority-based collection and dispatch of vectored interrupts

Design Justification

A priority queue combined with a FOR loop (collection), nested WHILE loops (servicing), and a modular ISR function together model vectored interrupt dispatch. Re-checking for a new higher-priority interrupt inside the servicing loop reflects real vectored/nested-interrupt behaviour, and the outer WHILE ensures the system keeps monitoring indefinitely rather than terminating after one interrupt.

Question 3 : Dynamic Memory-Mapped vs I/O-Mapped Selection

Problem Understanding

Input: per-device latency requirement, bandwidth requirement, current available address space, and current bus load. Output: a chosen I/O technique (memory-mapped or I/O-mapped) applied per device. Constraint: the decision must adapt at run time as bus and memory conditions change, not be fixed at design time.

Pseudocode

```
ALGORITHM Select_IO_Technique(device, THRESHOLD_LOW, THRESHOLD_HIGH)
BEGIN
    latency_req   = device.latency_requirement
    bandwidth_req = device.bandwidth_requirement
    free_space    = QUERY_available_address_space()

    IF latency_req < THRESHOLD_LOW OR bandwidth_req > THRESHOLD_HIGH THEN
        // High-performance devices need direct, wide instruction access
        IF free_space >= device.required_map_size THEN
            technique = MEMORY_MAPPED_IO
        ELSE
            // Not enough address space to map -> fall back
            technique = IO_MAPPED_IO
            LOG warning: 'insufficient address space, using I/O-mapped'
        END IF
    ELSE
        // Low-demand device: keep memory space free
        technique = IO_MAPPED_IO
    END IF

    CONFIGURE bus_decoder WITH technique FOR device
    RETURN technique
END

ALGORITHM Runtime_Reevaluation(device_list, INTERVAL)
BEGIN
    WHILE system_running DO
        FOR EACH device IN device_list DO
            new_choice = Select_IO_Technique(device, THRESHOLD_LOW, THRESHOLD_HIGH)
            IF new_choice != device.current_technique THEN
                SWITCH device TO new_choice   // reconfigure dynamically
                device.current_technique = new_choice
            END IF
        END FOR
        SLEEP(INTERVAL)
    END WHILE
END
```

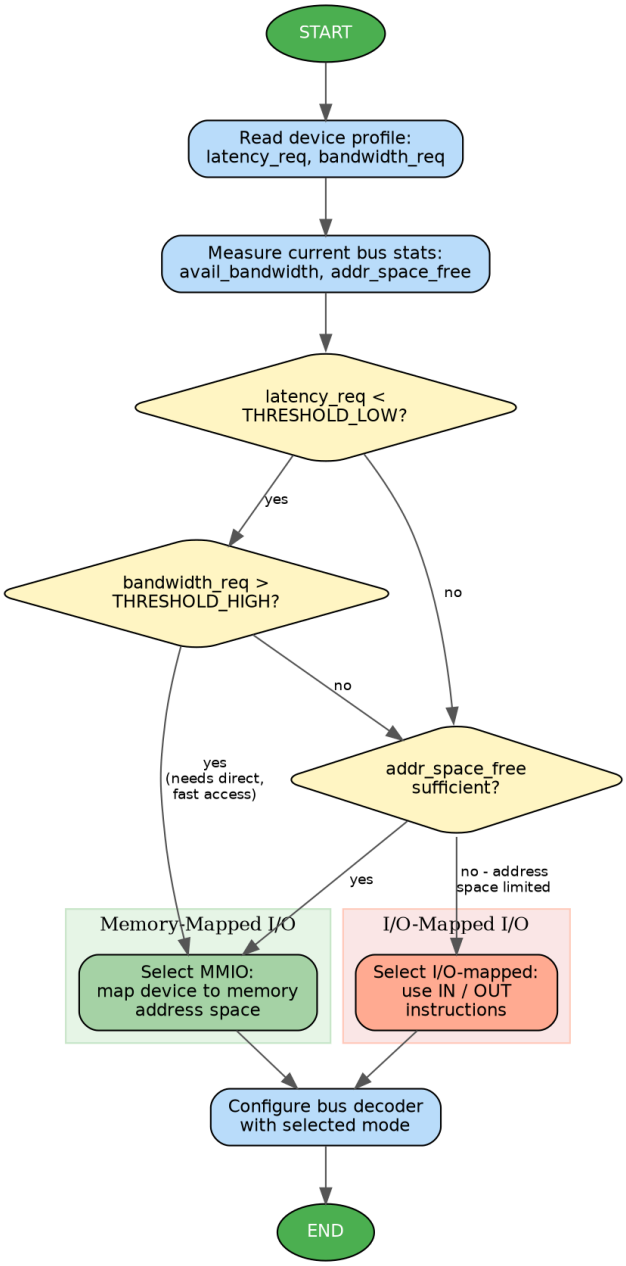

Fig 3.1 — Dynamic selection between memory-mapped and I/O-mapped access

Design Justification

Nested IF-ELSE structures evaluate latency, bandwidth and address-space constraints together, while the outer WHILE/FOR loop in Runtime_Reevaluation makes the choice dynamic rather than static, directly meeting the requirement to select 'dynamically based on latency and bandwidth'. The address-space fallback path also handles the edge case where MMIO is preferred but cannot be granted.

Question 4 : Bus Arbitration Across PCIe, USB and Thunderbolt

Problem Understanding

Input: bus requests from multiple devices on different interfaces (PCIe, USB, Thunderbolt), each with an interface priority class. Output: an ordered grant sequence giving each requesting device exclusive bus access in turn. Constraint: high-priority interfaces (e.g., PCIe/Thunderbolt for real-time transfer) must not starve lower-priority USB devices indefinitely.

Pseudocode

```
ALGORITHM Bus_Arbitration(request_queue, PRIORITY_MAP)
BEGIN
    round_robin_pointer = 0

    WHILE system_running DO
        FOR EACH device IN active_devices DO
            IF device.BUS_REQUEST == TRUE THEN
                ENQUEUE(request_queue, device)
            END IF
        END FOR

        IF request_queue IS EMPTY THEN
            CONTINUE   // bus stays idle
        END IF

        // Rank by priority with aging to prevent starvation

IF any device in request_queue has waited longer than MAX_WAIT_TIME THEN

    selected_device = LONGEST_WAITING_DEVICE

ELSE

    SORT request_queue BY PRIORITY_MAP[device.interface_type] DESCENDING,

                      round_robin_pointer


selected_device = request_queue.FIRST

END IF

        ASSERT GRANT TO selected_device
        LOCK bus FOR selected_device

        TRANSFER data OVER bus FOR selected_device

        WAIT UNTIL selected_device.BUS_RELEASE == TRUE
        UNLOCK bus

        REMOVE selected_device FROM request_queue
        round_robin_pointer = (round_robin_pointer + 1) MOD num_devices
    END WHILE
END

FUNCTION PRIORITY_MAP(interface_type)
BEGIN
    IF interface_type == THUNDERBOLT THEN RETURN 3
    ELSE IF interface_type == PCIE THEN RETURN 2
```

```
    ELSE IF interface_type == USB THEN RETURN 1
    END IF
END
```

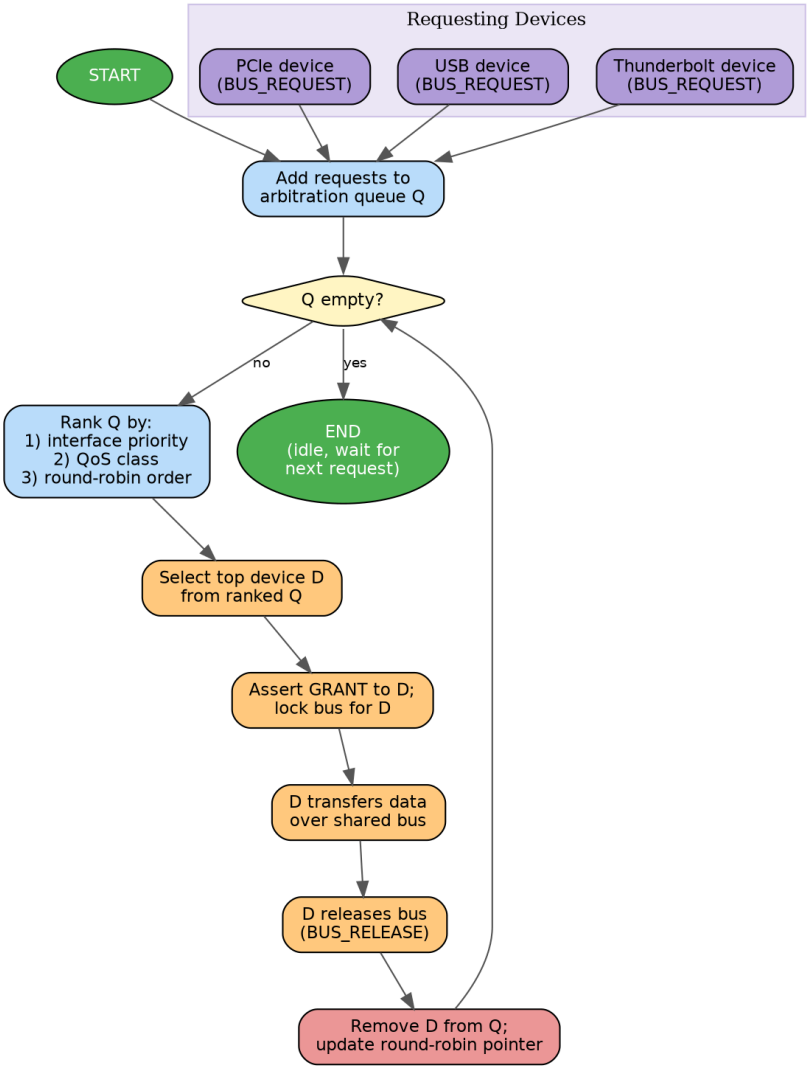

Fig 4.1 — Priority-and-round-robin bus arbitration for PCIe, USB and Thunderbolt

Design Justification

The FOR loop gathers requests each cycle, an IF guards the empty-queue edge case, and the round-robin pointer combined with priority ranking prevents starvation of lower-priority USB devices while still favouring latency-sensitive PCIe/Thunderbolt traffic. The PRIORITY_MAP function modularizes the priority rule so it can be extended to new interfaces without changing the core loop.

Question 5 : Hybrid I/O Communication (Polling + DMA/Interrupt)

Problem Understanding

Input: a list of devices, each with a rated transfer speed. Output: every device serviced using the technique appropriate to its speed class — polling for low-speed devices, DMA with interrupts for high-speed devices. Constraint: the algorithm must handle a mixed device set in a single pass and correctly branch per device rather than applying one technique globally.

Pseudocode

```
ALGORITHM Hybrid_IO_Communication(device_list, SPEED_THRESHOLD)
BEGIN
    FOR EACH device IN device_list DO
        IF device.speed <= SPEED_THRESHOLD THEN
            // ---- Low-speed path: Polling ----
            WHILE device.status_register != READY DO
                // busy-wait, simple and low overhead for slow devices
                NO_OP()
            END WHILE
            data = READ device.data_register
            PROCESS(data)

        ELSE
            // ---- High-speed path: DMA + Interrupt ----
            CONFIGURE DMA_channel(device.src, memory.dest, device.block_size)
            ISSUE DMA_START TO DMA_controller
            // CPU is free to continue with next device / other tasks

            REGISTER callback ON_DMA_COMPLETE FOR device
        END IF
    END FOR
END

ISR ON_DMA_COMPLETE(device)
BEGIN
    IF DMA.status_register == ERROR THEN
        RETRY_COUNT = RETRY_COUNT + 1
        IF RETRY_COUNT < MAX_RETRIES THEN
            RESTART DMA_channel FOR device
        ELSE
            LOG failure FOR device
        END IF
    ELSE
        MARK device.transfer_complete = TRUE
        CLEAR DMA interrupt flag
    END IF
END
```

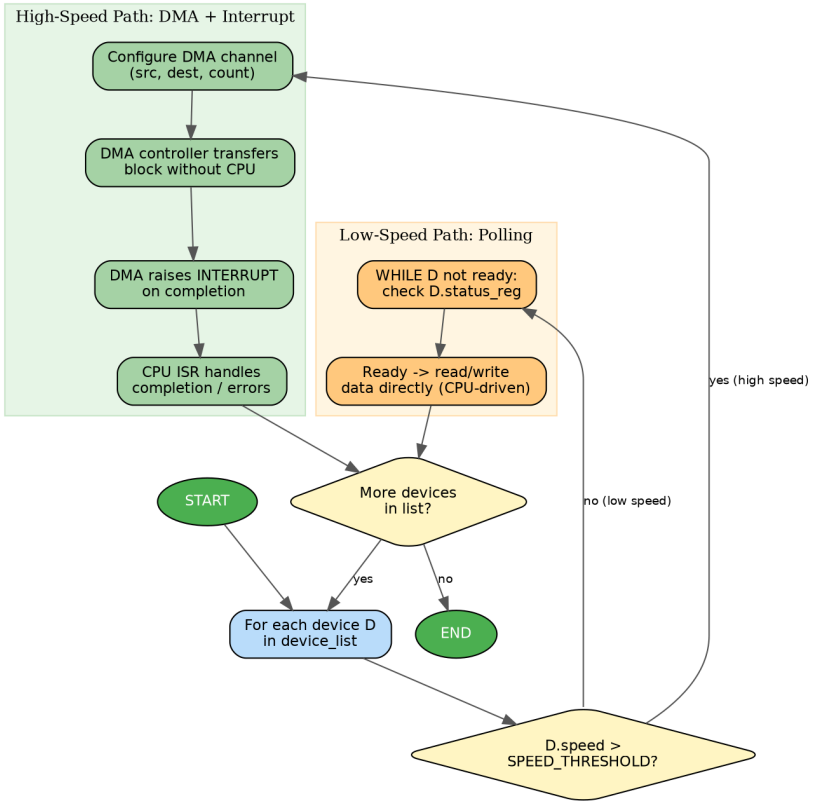

Fig 5.1 — Hybrid mechanism branching between polling and DMA/interrupt paths

Design Justification

A single FOR loop scans all devices, and an IF-ELSE branches each device into the polling path (WHILE busy-wait) or the DMA path (asynchronous, interrupt-driven), so both mechanisms are unified under one algorithm as required. The ISR includes a bounded retry counter, which handles the completeness criterion of dealing with transfer errors rather than only the success case.